

AI-Powered Real-Time Road Safety & Accident Detection System

End-to-End Technical Guide & System Blueprint
Architecture, Workflow, File Mapping, and Pretrained Model Usage

1. System Architecture & End-to-End Pipeline

The Accident Detection System processes raw road and CCTV footage (both accident sequences and normal driving) through a 4-tier computer vision and deep learning pipeline. The architecture seamlessly combines real-time object detection and tracking with temporal sequence classification:

Stage	Core Technology	Primary Purpose	Output Data
1. OpenCV Preprocessing	OpenCV (cv2)	Frame ingestion, uniform sampling (16 frames per video clip)	Normalized sequences (224x224, 32 frames)
2. YOLO Detection	YOLOv11 nano	Detects vehicles (cars, motorcycles, buses, trucks, bicycles, and pedestrians), confidences, class probabilities	Bounding boxes, class IDs, and confidences
3. Object Tracking	ByteTrack + Kinematics	Maintains persistent IDs across occlusion and distance changes	Track IDs, position displacements, trajectories
4. CNN-LSTM Classifier	ResNet18 + LSTM	Extracts spatial feature embeddings and analyzes temporal dynamics	Accident vs. normal driving classification score

2. File Mapping — Which Files Belong to What

The project is structured into clear functional domains. Below is the precise mapping of files to each pipeline stage:

Component	File Path	Specific Responsibility
1. OpenCV Preprocessing	preprocessing/video_processor.py	Inspects FPS, dimensions, frame count; writes standardized MP4.
	preprocessing/video_preprocessing.py	Extracts, resizes, and normalizes sequences for DL model input.
	preprocessing/frame_extraction.py	Uniform temporal sampling (exactly 16 frames per video clip).
	preprocessing/resize_frames.py	Standardizes all frames to 224x224 pixels.
	preprocessing/normalize_frames.py	Applies ImageNet mean/std normalization and tensor conversion.
	preprocessing/optical_flow.py	Computes Farneback dense motion energy between frames.
	preprocessing/utils.py	Helper functions for video metadata extraction.
2. YOLO Detection	detection/yolo_detector.py	Core YOLODetector class wrapping YOLOv11 for road objects.
	detection/weights/yolo11n.pt	Pretrained YOLOv11 nano model weights.
	detection/detection_config.py	Confidence thresholds (0.40) and target road classes.
	detection/draw_bounding_boxes.py	Draws labels, boxes, and confidence banners on video frames.
	detection/detect_objects.py	Convenience functional wrapper for single-frame detection.
	detection/run_detection.py	CLI execution script for Person 1 detection.

3. Object Tracking	tracking/tracker.py	MultiObjectTracker integrating ByteTrack frame association.
	tracking/trajectory.py	TrajectoryTracker managing rolling center points and trails.
	tracking/motion_features.py	Computes speed (m/s), direction (Up/Down/Left/Right), anomaly alerts.
	tracking/speed_estimation.py	Calibrated conversion from pixels/second to meters/second.
	tracking/tracking_config.py	Tracker configuration parameters (history length = 30).
	tracking/run_tracking.py	CLI execution script for Person 2 tracking deliverables.
4. CNN-LSTM Classifier	models/cnn_model.py	Pretrained ResNet18 spatial feature extractor (512-dim output).
	models/lstm_model.py	2-layer LSTM temporal network with dropout and classification head.
	models/cnn_lstm_model.py	Composite end-to-end model (B, 16, 3, 224, 224) -> Probability.
	models/saved_models/best_model.pth	Trained model checkpoint for immediate inference.
	training/dataset_loader.py	PyTorch VideoSequenceDataset yielding sequence tensors and labels.
	training/train_cnn_lstm.py	Training loop with Adam optimizer and validation tracking.
	training/loss_functions.py	Weighted BCE loss prioritizing accident recall.
	inference/accident_predictor.py	High-level prediction API returning accident verdict & confidence.
	inference/video_inference.py	Renders output video with HUD probability gauge & status bar.
Integration & Entry	integration/accident_detection_pipeline.py	Full end-to-end pipeline connecting Stages 1 -> 2 -> 3 -> 4.
	main.py	Top-level user CLI entrypoint (python main.py --video <path>).

3. Deep Technical Breakdown: What is Used & How It is Used

Stage 1: OpenCV Preprocessing

What is used: OpenCV (cv2.VideoCapture, cv2.resize, cv2.cvtColor), NumPy contiguous arrays, and PyTorch tensor transformations.

How it is used: Raw road videos often have varying resolutions (1080p, 720p, 360p) and frame rates (10 to 30 FPS). The preprocessor inspects video properties and extracts a uniform temporal sample of exactly 16 frames spanning the clip duration. Each frame is resized to 224x224 pixels and transformed using ImageNet normalization (Mean: [0.485, 0.456, 0.406], Std: [0.229, 0.224, 0.225]). This ensures consistent tensor dimensions (1, 16, 3, 224, 224) ready for deep neural network consumption.

Stage 2: YOLO Road Object Detection

What is used: Pretrained YOLOv11 nano model weights (detection/weights/yolo11n.pt) powered by Ultralytics.

How it is used: Each frame is fed through the YOLO network with a confidence threshold of 0.40. The model is restricted specifically to road safety classes: car, motorcycle, bus, truck, bicycle, and person. Detections provide pixel bounding boxes (x1, y1, x2, y2), class names, and confidence probabilities. Irrelevant background objects (such as trees, signs, animals) are filtered out.

Stage 3: Multi-Object Tracking & Motion Analysis

What is used: ByteTrack multi-object tracking algorithm, Euclidean centroid computation, and kinematic motion feature estimation.

How it is used: YOLO alone only detects objects per frame independently without remembering identities. ByteTrack assigns persistent numerical IDs (e.g. ID: 1, ID: 2) across consecutive frames, even during partial occlusions. For each tracked vehicle, the system computes:

- **Centroid & Trails:** Centers (cx, cy) stored in a rolling 30-frame deque to draw visible trajectory lines.
- **Displacement:** Frame-to-frame pixel movement: $\Delta d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.
- **Speed:** Speed in m/s using actual video FPS: $v = (\Delta d \times \text{FPS}) / \text{pixels_per_meter}$.
- **Direction:** Cardinal movement: 'Up', 'Down', 'Left', 'Right', or 'Stationary'.
- **Kinematic Alerts:** Flags sudden direction reversals, abnormal speed spikes, and rapid deceleration.

Stage 4: CNN-LSTM Temporal Accident Classifier

What is used: Pretrained ResNet18 CNN backbone + 2-layer LSTM recurrent network + Sigmoid classification head.

How it is used: An accident is a temporal event that unfolds over time (e.g., normal approach -> sudden collision/impact -> post-crash halt). Single-frame image models cannot distinguish an accident from a parked car. The hybrid CNN-LSTM solves this:

1. **CNN Spatial Feature Extractor:** The 16 frames are passed through the frozen ResNet18 backbone. The final classification layer is removed, yielding a 512-dimensional feature vector per frame: (B, 16, 512).
2. **LSTM Temporal Aggregator:** The sequence of 16 vectors is fed into the 2-layer LSTM (Hidden dim: 128). The LSTM learns how visual features change over time.
3. **Classification Head:** The final hidden representation passes through a Linear layer with Sigmoid activation, outputting the probability of an accident $P \in [0.0, 1.0]$. If $P \geq 0.50$, the verdict is **ACCIDENT DETECTED**;

otherwise **NORMAL TRAFFIC**.

Why We Use a Pretrained Model (The Core Engineering Decision):

- **CNN Spatial Backbone:** We use a **pretrained ResNet18** (trained on ImageNet). Training a deep CNN from scratch requires hundreds of thousands of road images and weeks of GPU compute. A pretrained CNN already understands low-level and high-level visual features (edges, contours, vehicle shapes, road textures). Freezing this backbone allows rapid feature extraction even on standard laptops without GPUs.
- **LSTM Sequence Model:** While the CNN backbone is pretrained, the **LSTM head is trained specifically on accident vs. normal video sequences**. This gives you the best of both worlds: high-level visual features from ImageNet transfer learning combined with customized temporal accident detection.

4. How to Run & User Guide

Follow these exact steps to run the system:

Step 1: Activate Virtual Environment

```
.venv\Scripts\activate
```

Step 2: Run End-to-End Prediction on Any Video

```
To test on an accident video and get the final answer immediately:
python main.py --video data/raw/videos/accident/mock_accident.mp4
```

```
To test on a normal traffic video:
python main.py --video data/raw/videos/normal/real_road_test.mp4
```

Sample Terminal Output:

```
=====
ROAD SAFETY AI – FULL PIPELINE (OPENCV -> YOLO -> TRACKING -> CNN-LSTM)
=====
[Stage 1] OpenCV Preprocessing...
Input: data/raw/videos/accident/mock_accident.mp4
Properties: 640x360 @ 10.0 FPS | 60 frames (~6.0s)
[Stage 2 & 3] YOLO Detection & Object Tracking...
Unique entities tracked: 2
[Stage 4] CNN + LSTM Temporal Accident Classifier...
=====
FINAL ACCIDENT PREDICTION VERDICT
=====
VERDICT: ACCIDENT DETECTED
ACCIDENT PROBABILITY: 97.86%
IS ACCIDENT: True
OUTPUT VIDEO: outputs/predictions/predicted_mock_accident.mp4
=====
```

5. Deliverables & Output Verification

When the pipeline executes, the following deliverables are saved in the outputs/ folder:

Output Folder	Output File	Description
outputs/predictions/	predicted_<video>.mp4	Video displaying bounding boxes, tracking IDs, and real-time HUD accident
outputs/tracked_videos/	tracked_video.mp4	Annotated video with persistent IDs, velocities in m/s, and historical moti
outputs/tracked_videos/	trajectories.csv	Full tabular kinematics dataset (frame, timestamp, track ID, speed, direct
outputs/tracked_videos/	tracking_report.txt	Text summary report detailing unique vehicles, maximum speeds, and d
models/saved_models/	best_model.pth	Trained PyTorch weights checkpoint for the CNN+LSTM accident classif